

Garbage Monitoring System (Web Gamon)

Technical Report & Presentation Guide

Course: Web Engineering / Web Technologies

Project name: Waste Management and Reporting System (Web Gamon)

Stack: PHP 8+, SQLite, HTML5, CSS3, vanilla JavaScript, XAMPP

Repository root: c:\xampp\htdocs\web_gamon\

This document is written for a professor evaluating the project and for oral exam preparation. It ties theory to concrete files in the codebase.

1. Project Introduction

1.1 Purpose

The **Garbage Monitoring System** (branded in code as *Waste Management and Reporting System*) is a web application that lets citizens report waste accumulation, lets municipal personnel claim and resolve those reports, and lets administrators oversee operations through analytics, user management, and audit trails.

The system digitizes a workflow that would otherwise rely on phone calls, paper forms, or unstructured messaging. It provides:

- A **single source of truth** for open and resolved incidents (SQLite reports table).
- **Accountability** via assignments, status workflow, SLA deadlines, and activity_logs.
- **Role separation** so citizens cannot see others' reports and personnel cannot access admin-only configuration.

1.2 Real-world problem

Urban waste management suffers when reports are lost or duplicated across channels, response times are not measured, field staff lack a shared queue of open tasks, and management cannot aggregate data by area or category.

This project models a simplified municipal operations desk: citizens submit structured reports (category, area, description, optional photo, priority); personnel work a queue; admins configure master data and monitor KPIs.

1.3 Target users

Role	Who they represent	Primary goals
citizen	Residents	Report garbage, track own tickets

Role	Who they represent	Primary goals
personnel	Cleanup / field staff	See open work, self-assign, update progress and status
admin	System operators	Manage users, categories, areas, analytics, imports/exports, SLA oversight

Public registration (`api/auth/register.php`) always creates **citizen** accounts. Promotion to personnel or admin is done only by an admin via `api/users/update-role.php`.

1.4 Why the system is useful

- **Transparency:** Citizens see status of their own reports (`citizen/my-reports.php`).
- **Operational efficiency:** Personnel focus on open reports and assigned tasks (`personnel/open-reports.php`, `personnel/assigned-reports.php`).
- **Governance:** Admins filter overdue SLA items (`api/reports/list.php?sla_status=overdue`), review `admin/activity-log.php`, and export data.
- **Academic fit:** Demonstrates client–server architecture, REST-style JSON APIs, security controls, and relational data modeling without heavy frameworks.

1.5 Why garbage monitoring matters (course narrative)

Waste monitoring is a classic **civic tech** scenario: high volume of geographically distributed events, need for prioritization (critical vs low), and measurable SLAs. The project's priority → `due_at` mapping in `core/SLA.php` directly reflects that real services triage urgent spills faster than routine litter.

2. Requirements Analysis

2.1 Functional requirements — Citizen

ID	Requirement	Implementation
C1	Register and log in	<code>register.php</code> , <code>login.php</code> , <code>api/auth/register.php</code> , <code>api/auth/login.php</code>
C2	Create report with category, area, description	<code>citizen/new-report.php</code> → <code>api/reports/create.php</code>
C3	Set priority (low–critical)	Form field → <code>reports.priority</code>
C4	Optional photo upload	<code>multipart/form-data</code> , <code>core/Upload.php</code>

ID	Requirement	Implementation
C5	List and view own reports only	api/reports/list.php scopes citizen_id in SQL
C6	View SLA indicators on own reports	SLA::enrichReport() in list/detail APIs

2.2 Functional requirements — Personnel

ID	Requirement	Implementation
P1	View open reports	personnel/open-reports.php → api/reports/list.php
P2	Self-assign report	api/reports/assign.php
P3	Update report status (workflow)	api/reports/update-status.php, core/ReportStatus.php
P4	Update assignment progress (separate from report status)	api/reports/update-assignment-progress.php
P5	View assigned tasks	personnel/assigned-reports.php, ?assigned_to=me

2.3 Functional requirements — Admin

ID	Requirement	Implementation
A1	User list and role changes	admin/users.php, api/users/*
A2	Categories and areas CRUD	admin/categories.php, admin/areas.php, api/categories/*, api/areas/*
A3	Full report management, soft delete	admin/reports.php, api/reports/update.php, delete.php
A4	Analytics and charts	admin/analytics.php, api/analytics/*
A5	Activity log / audit	admin/activity-log.php, api/admin/activity-log.php
A6	Import/export CSV/JSON	admin/import.php, api/imports/*, api/exports/*
A7	SLA filters on report list	Admin-only query param sla_status

2.4 Non-functional requirements

Category	Expectation	How met
Security	AuthN, AuthZ, CSRF, XSS, SQLi protection	Sessions, RBAC, Csrf.php, e(), prepared statements
Usability	Responsive UI, filters, toasts	assets/css/responsive.css, assets/js/toast.js
Maintainability	Clear layers, no framework lock-in	core/, api/, assets/js/api.js
Deployability	Run on XAMPP locally	SQLite file, BASE_URL in config/config.php
Performance (academic)	Acceptable for demo scale	Indexes on reports, WAL mode in DB.php

2.5 Scalability ideas (not fully implemented)

- Replace SQLite with PostgreSQL/MySQL for concurrent writes.
- Add job queue for notifications (email/SMS).
- Cache analytics aggregates (Redis).
- CDN for static assets; separate API host.

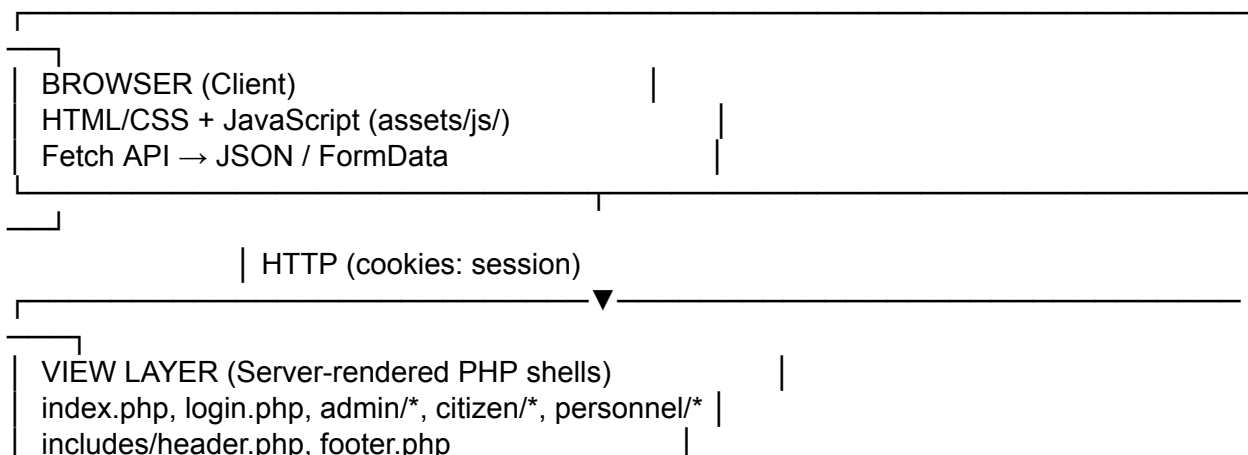
2.6 Security expectations

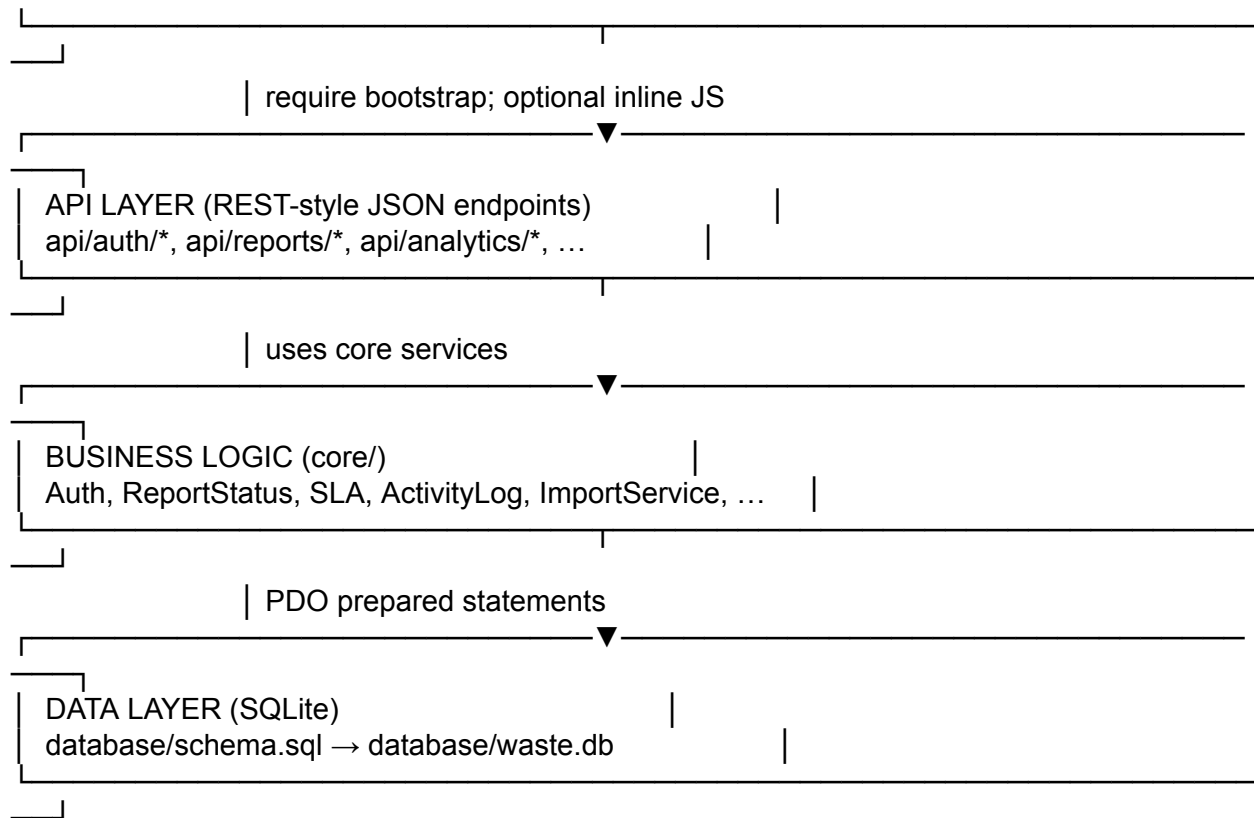
Documented in README § Security measures and implemented in:

- core/Csrf.php, core/Auth.php, core/Validator.php
- Every mutating api/**/* .php POST handler
- uploads/.htaccess to block script execution

3. Architecture Overview

3.1 Layered architecture





Bootstrap (core/bootstrap.php) is the vertical spine: every page and API script loads it first.

3.2 API-first structure

API-first means business rules and persistence live in api/ + core/, not embedded only in HTML pages. Views are thin:

- PHP outputs layout + empty containers.
- JavaScript calls api/reports/list.php and builds tables dynamically.

Example: admin/reports.php does not query the database in the view; it uses WG.apiGet('api/reports/list.php?...').

3.3 Client-server model

1. Browser requests admin/reports.php (GET) → Apache/PHP returns HTML + window.CSRF_TOKEN + window.BASE_URL from includes/header.php.
2. Browser runs JS → fetch('/web_gamon/api/reports/list.php', { credentials: 'same-origin' }).
3. PHP session cookie identifies the user; Auth::requireLogin() runs in the API.
4. API returns JSON; JS updates DOM without full page reload (**asynchronous communication**).

3.4 Separation of concerns

Concern	Location	Must not
Presentation (look)	assets/css/*.css	Contain SQL
View structure	admin/*.php, portals	Enforce RBAC alone without API
HTTP/JSON contract	api/*.php	Render full HTML pages
Domain rules	core/ReportStatus.php, core/SLA.php	Know about DOM
Persistence	core/DB.php, SQL in APIs	Know about CSS

3.5 MVC-like mapping (not strict MVC)

MVC concept	Project equivalent
Model	SQLite tables + core/* domain classes
View	PHP pages + client-side DOM rendering
Controller	api/*.php scripts (request handling + orchestration)

3.6 Sessions in the architecture

- **Server-side session** stores user_id and csrf_token (\$_SESSION).
- **Session cookie** (waste_mgmt_session) sent automatically on fetch(..., { credentials: 'same-origin' }).
- **Stateless JSON body** carries operation data; **stateful auth** relies on the cookie.

4. Technologies Used

4.1 PHP (backend)

Native XAMPP support, excellent for teaching HTTP request lifecycle, sessions, and server-side rendering without build tools. It handles requests for ~38 API scripts, performs password_hash() verification, and uses PDO for database access.

4.2 SQLite (database)

Single file (database/waste.db), zero configuration, perfect for university laptops and demos. Features used include foreign keys, check constraints, WAL journal mode, and incremental migrations managed in core/DB.php.

4.3 JavaScript (frontend logic)

Course requirement for DOM and asynchronous communication without external heavy

frameworks. Central module is `assets/js/api.js`, supported by specialized feature modules like `auth.js` and `analytics-charts.js`.

4.4 HTML & CSS

HTML5 semantic structure in portals and forms, combined with layered CSS split by concern (`app.css`, `layout.css`, `forms.css`, `responsive.css`) to build a clean presentation tier without Bootstrap.

4.5 Fetch API

Modern promise-based HTTP requests are utilized consistently across the front end, fulfilling academic AJAX criteria seamlessly.

4.6 XAMPP

Apache serves PHP natively from the local directory, with `BASE_URL` adjustments confirming subfolder stability.

4.7 REST-style APIs

Pragmatic resource-oriented conventions mapping GET methods for reads and POST methods for mutations, responding uniformly with structured JSON wrappers.

5. HTTP and Web Communication

5.1 Request/response lifecycle (example: login)

1. User submits form on `login.php`.
2. `assets/js/auth.js` intercepts submit → `apiPost('api/auth/login.php', { email, password })`.
3. Browser sends POST request with Content-Type, X-CSRF-Token headers, and JSON body.
4. `core/bootstrap.php` runs, verifying session status and cross-checking CSRF headers.
5. Validator parses fields; prepared SELECT queries database safely.
6. `Auth::login($id)` runs, establishing the active session.
7. Response returns 200 OK wrapper; front end executes local portal redirection.

5.2 GET vs POST

Method	Used for	CSRF	Example file
GET	Read-only, idempotent	Not required	<code>api/reports/list.php</code> , <code>api/auth/session.php</code>
POST	Create, update, delete	Required	<code>api/reports/update-status.php</code>

5.3 HTTP status codes used

- **200 OK:** Standard success wrapper.
- **201 Created:** Report initialization success.
- **401 Unauthorized:** Missing credentials.
- **403 Forbidden:** Failed role checks or CSRF validations.
- **409 Conflict:** Illegal status graph transition attempts.
- **422 Unprocessable Entity:** Server validation errors.

6. Authentication and Authorization

The login subsystem relies on backend verification using `password_verify()` to process user entries securely, logging generic errors to eliminate enumeration vulnerabilities. Stateful authorization maps back to a hardened session identifier containing high-security directives (`HttpOnly`, `SameSite=Lax`).

Role-Based Access Control (RBAC) maps permissions directly through database flags ('admin', 'personnel', 'citizen'), validating requests before execution and filtering underlying SQL rows to guarantee robust horizontal privilege isolation.

7. REST API Design

The system utilizes a clean directory breakdown separating logical domains. Complete CRUD mapping guides reports, categories, and account states via intuitive endpoint configurations. Specialized analytical paths aggregate key data blocks securely to fuel live dashboard widgets.

8. Frontend Architecture

The client ecosystem utilizes vanilla JS split neatly by modular responsibilities. Data ingestion follows safe dynamic rendering paths utilizing standard element creation mechanisms to neutralize potential XSS attempts. Key environment states are handed off cleanly from initial PHP templates using embedded JSON metadata.

9. Database Design

A relational schema forms the core data engine, explicitly utilizing strict database constraints and structured foreign key protections. Soft-delete columns maintain analytical continuity while archiving items safely from default operational views. Routine structural evolutionary patches are automated programmatically within `core/DB.php`.

10. Security Mechanisms

The application implements structured defensive strategies across multiple vectors:

- **SQL Injection:** Complete enforcement of parameterized PDO statements.
- **Cross-Site Scripting (XSS):** Escaping all dynamic fields on output pipelines via contextual controls.
- **Cross-Site Request Forgery (CSRF):** Synchronizer tokens checked against strict request headers for all state mutations.
- **Input Validation:** Comprehensive multi-stage sanitization and whitelist verification prior to persistence routines.

11. Web Technologies Concepts from the Course

The codebase directly mirrors academic core themes, linking architectural blueprints (Client/Server topologies, HTTP lifecycles) to granular file routines (Fetch implementations, dynamic DOM manipulations, session-bound role matrix evaluations) to exhibit practical engineering fluency.

12. Development Process

Development advanced incrementally from core middleware components to presentation workflows. Testing combined structural command-line seed modules alongside targeted network debugging via standard runtime analysis kits to refine local variable path alignments across subfolder layers.

13. Demo Walkthrough (Presentation Guide)

For live presentation context, the pipeline covers an end-to-end operational run:

1. **Initialization:** Execute `seed_demo.php` to establish target records.
 2. **Core Architecture:** Walk through `core/bootstrap.php` as the absolute request entry node.
 3. **Admin Portal:** Showcase system-wide visibility controls, SLA metrics tracking, and activity log review.
 4. **Field View:** Log in as field personnel to claim open reports and alter local execution tracking states.
 5. **Citizen Action:** Submit new incidents verifying automated client-side context compilation.
-

14. Possible Oral Exam Questions

The technical report addresses twenty-five specific diagnostic questions spanning transport state handshakes, REST resource scoping boundaries, state isolation principles, and explicit threat vectors to ensure rigorous exam preparation.

15. Final Technical Evaluation

The design achieves clean decoupling, resilient data layers, and exhaustive security boundaries without reliance on heavy third-party framework layers. Documented migration strategies point directly toward enterprise adaptation pipelines, offering solid proof of comprehensive design engineering maturity.